

微服务的应用与分析

一、Saga

Saga 是一种处理分布式事务的模式，旨在确保一系列操作要么全部成功执行，要么在失败时能够全部回滚，以保持数据的一致性。

1、基本原理

Saga 事务由一系列有序的子事务组成，每个子事务都支持幂等操作。
当某个子事务失败时，Saga 会触发之前成功执行的子事务的补偿操作，以确保数据的一致性。

2、应用场景

Saga 模式广泛应用于分布式系统中，如电商平台中的订单支付、物流配送、酒店预订等场景，其中涉及多个服务的协同操作。

3、实现方式

Saga 可以通过基于协作或基于编排两种方式实现。
基于协作时，每个子事务知道下一个应执行的子事务，并通过消息传递进行通信。
基于编排时，有一个中心协调者负责调度和协调各个子事务的执行。

二、服务编排

服务编排是指将多个服务按照一定的逻辑关系进行组合、调度和执行的过程，以解决分布式系统中的集成问题。常见的编排模型包括流水线模型、分支模型和迭代模型。

服务编排有如下三种操作

- 服务选择：根据用户需求和系统功能选择合适的服务。
- 服务组合：将选定的服务按照业务逻辑进行有序组合。
- 服务调度：决定每个服务的调用时机和调用顺序。

三、事件溯源

事件溯源是一种记录和存储应用程序状态变化的设计模式，通过持久化存储每个状态变化作为事件来重建系统的历史状态和进行数据恢复。

1、核心思想

将系统中的每次状态变化表示为事件，并将这些事件持久化存储。通过重放这些事件来重新构建系统的状态。

2、关键组成部分：

事件：系统中发生的有意义的状态变化。

事件存储：持久化存储所有事件的系统。

聚合根：负责接收和应用事件，确保事件按正确顺序应用以维护一致性。

四、在线旅游预订平台场景

假设有一个在线旅游预订平台，用户可以在该平台上预订机票、酒店和租车等服务。以下是如何在这个场景中运用 Saga、服务编排和事件溯源的详细描述。

1.、Saga 的应用：

当用户在平台上预订一个完整的旅行套餐（包括机票、酒店和租车）时，系统会启动一个 Saga 事务来确保所有预订操作要么全部成功，要么在出现问题时能够全部回滚。

这个 Saga 事务可能包含以下子事务：预订机票、预订酒店和预订租车。

如果在预订过程中任何一个子事务失败（例如酒店可能没有可用房间），Saga 会触发之前成功预订服务的补偿事务。比如，如果已经成功预订了机票，但酒店预订失败，那么 Saga 会触发机票退订的补偿事务。

2.、服务编排的应用：

在这个场景中，服务编排负责组织和调度机票预订服务、酒店预订服务和租车预订服务的调用。

服务编排确保这些服务按照正确的顺序被调用，比如在确认机票预订成功后才进行酒店和租车的预订。

如果某个服务调用失败，服务编排会负责协调回滚操作，确保整个旅行套餐的预订状态保持一致。

3、事件溯源的应用：

在预订过程中，每个关键步骤（如机票预订、酒店预订和租车预订）都会作为事件被记录下来，并存储在一个不可变的事件日志中。

这些事件包括“机票预订成功事件”、“酒店预订失败事件”等，每个事件都包含详细的时间戳和数据。

通过事件溯源，平台可以重建用户的预订历史，以便进行故障排查、数据分析或提供客户服务。

此外，其他服务也可以订阅这些事件来触发相应的操作。例如，当用户成功预订机票后，可以触发一个通知服务来提醒用户预订已成功。

4、场景总结

在这个在线旅游预订平台的场景中，Saga 可以确保分布式事务的一致性和原子性，服务编排负责组织和调度各个服务的调用顺序，而事件溯源则提供了对历史事件的重建和分析能力。这三者的结合使得平台能够高效地处理复杂的预订流程，并在出现问题时迅速恢复状态。